BANCO DE DADOS · AULA 05

Transações e Concorrência

Propriedades ACID e Gerenciamento de Transações · Controle de Concorrência, Níveis Concorrência, Níveis de Isolamento, Locks e Deadlocks.

ACID | ISOLATION LEVELS | LOCKS & DEADLOCKS



SUMÁRIO

Agenda da Aula

Dois blocos cobrindo teoria ACID e prática de concorrência em produção.

5.1

Propriedades ACID e Gerenciamento de Transações

- ›O que é uma transação
- ›Propriedades ACID (Atomicity, Consistency, Isolation, Durability)
- ›BEGIN TRAN / COMMIT / ROLLBACK
- ›Transações explícitas, implícitas e autocommit
- ›SAVEPOINT e rollback parcial
- ›Transações aninhadas
- ›Transações distribuídas (MSDTC)
- ›Padrão profissional com TRY/CATCH

5.2

Controle de Concorrência, Níveis de Isolamento, Locks e Deadlocks

- ›Concorrência: o problema
- ›Tipos de locks (S, X, IS, IX)
- ›Matriz de compatibilidade
- ›Níveis de isolamento (READ UNCOMMITTED a SERIALIZABLE)
- ›Snapshot isolation (RCSI)
- ›Deadlocks — causa e diagnóstico
- ›Padrões de prevenção
- ›Checklist profissional

SEÇÃO

5.1

✦ CHAPTER

Propriedades ACID e Gerenciamento de Transações

A fundação teórica que garante integridade de dados em qualquer SGBD relacional.

01 → O que é uma transação

02 → Propriedades ACID

03 → BEGIN/COMMIT/ROLLBACK

04 → Modos de transação

05 → SAVEPOINT

06 → Transações aninhadas

07 → MSDTC

08 → Padrão TRY/CATCH + XACT_STATE

CONCEITO FUNDAMENTAL

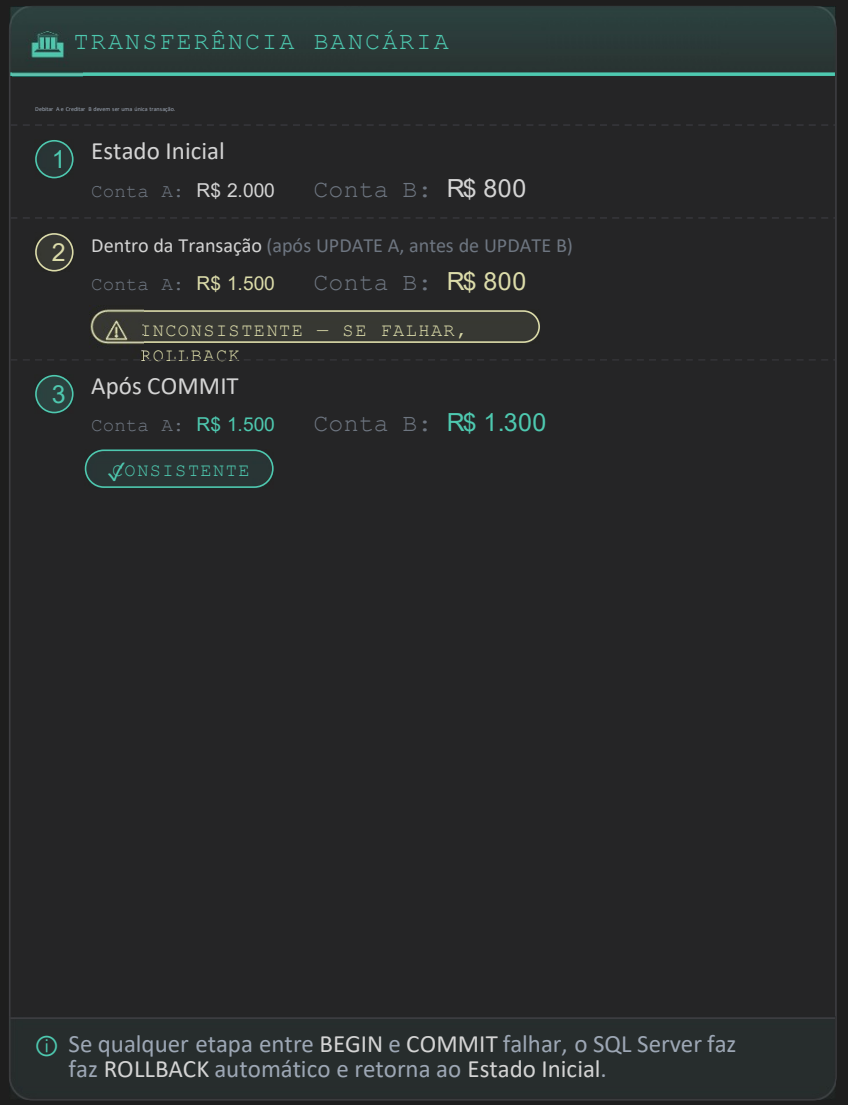
O que é uma Transação?

Uma transação é uma **unidade lógica de trabalho** composta por uma ou mais operações que devem ser executadas de forma **atômica** — ou todas as operações são confirmadas (COMMIT) ou nenhuma é (ROLLBACK). Não existe meio-termo: uma transação parcial é inaceitável.

O exemplo clássico é uma **transferência bancária**: debitar R$ 500 da conta A e creditar R$ 500 na conta B são duas são duas operações que devem ser tratadas como uma unidade. Se a segunda falhar (queda de energia, erro de erro de constraint, deadlock), a primeira deve ser desfeita — caso contrário, o dinheiro "desapareceria" do sistema. sistema.

CARACTERÍSTICAS ESSENCIAIS

- ›Unidade lógica de trabalho (Logical Unit of Work)
- ›Atômica e indivisível
- ›Pode conter uma ou mais instruções DML INSERT, UPDATE, DELETE



TEORIA FUNDAMENTAL

Propriedades ACID

As quatro garantias que um SGBD relacional deve oferecer para toda transação.

4 GARANTIAS · 1 PILAR
INTEGRIDADE TRANSACIONAL

A — Atomicity (Atomicidade)

Tudo ou nada. A transação é uma unidade indivisível — ou todas as operações operações são confirmadas (COMMIT) ou nenhuma é (ROLLBACK).

✓ COMO O SQL SERVER GARANTE

Transaction Log (WAL — Write-Ahead Log) ROLLBACK automático
SET XACT_ABORT ON

C — Consistency (Consistência)

A transação leva o banco de um estado válido para outro estado válido. Constraints, triggers e regras de negócio nunca podem ser violadas.

✓ COMO O SQL SERVER GARANTE

CHECK constraints, FOREIGN KEYS, triggers, DEFAULT values, NULL/NOT NULL, data types.

I — Isolation (Isolamento)

Transações concorrentes não interferem entre si. O resultado deve ser equivalente a alguma execução serial das transações.

✓ COMO O SQL SERVER GARANTE

Locks (S, X, IS, IX, SIX) Níveis de isolamento READ UNCOMMITTED
SERIALIZABLE MVCC

D — Durability (Durabilidade)

Uma vez confirmada (COMMIT), a transação persiste mesmo em caso de queda de queda de energia, falha de hardware ou crash do sistema.

✓ COMO O SQL SERVER GARANTE

Write-Ahead Log (WAL) ANTES
redo phase

→ ACID é a base teórica — vamos ver agora como o SQL Server implementa cada uma destas garantias na prática.

SINTAXE BÁSICA

BEGIN, COMMIT e ROLLBACK

T-SQL oferece três comandos fundamentais para controle de transações explícitas. Cada um tem papel específico e deve ser usado em padrões bem definidos.

✦ COMANDOS

- › BEGIN TRANSACTION (ou BEGIN TRAN) — inicia uma transação explícita
- › COMMIT TRANSACTION (ou COMMIT) — confirma, tornando alterações permanentes
- › ROLLBACK TRANSACTION (ou ROLLBACK) — desfaz todas as alterações desde BEGIN
- › SAVE TRANSACTION nome — cria ponto de salvamento parcial

◆ BOAS PRÁTICAS

- › Sempre nomear transações: BEGIN TRAN Transferencia
- › Usar TRY/CATCH para capturar erros
- › Verificar XACT_STATE() antes de ROLLBACK no CATCH
- › SET XACT_ABORT ON — auto-rollback em erros de runtime
- › Nunca deixar transação aberta (causa locks!)

✦ ESTADOS DE TRANSAÇÃO

- **Ativa** (após BEGIN, antes de COMMIT/ROLLBACK)
- **Confirmada** (após COMMIT)
- **Desfeita** (após ROLLBACK)
- **Doomed** (erro grave, deve ROLLBACK)

⚠ XACT_ABORT ON É MANDATÓRIO

Sem `SET XACT_ABORT ON`, erros de runtime (violação de constraint) deixam a transação ativa — causando locks eternos em produção.

✦ TransferenciaBancaria.sql · T-SQL

```
-- Padrão profissional: TRY/CATCH + XACT_STATE
SET XACT_ABORT ON- auto-rollback em erros de runtime

BEGIN TRY
    BEGIN TRANSACTION

    -- 1. Débito na conta origem
    UPDATE
    SET                 500.00
    WHERE           1

    IF@@ROWCOUNT  0
        THROW50001  'Conta origem não encontrada'
                                                1

    -- 2. Crédito na conta destino
    UPDATE
    SET                 500.00
    WHERE           2

    IF@@ROWCOUNT  0
        THROW50002  'Conta destino não encontrada'
                                                1

    -- 3. Log da transferência
   INSERT INTO

    VALUES
         1 2 500.00 GETDATE

    -- Tudo certo — confirma
    COMMIT TRANSACTION
    PRINT'Transferência concluída com sucesso'
END TRY
BEGIN CATCH
    -- Verifica se há transação ativa
    IFXACT_STATE     0
        ROLLBACK TRANSACTION

    -- Log estruturado do erro
   INSERT INTO

    VALUES
       ERROR_NUMBER
        ERROR_MESSAGE
        ERROR_PROCEDURE
       ERROR_LINE
        GETDATE

    -- Re-lança o erro para o cliente
    THROW
END CATCH
GO
```

3 MODOS · 1 PADRÃO PROFISSIONAL

Modos de Transação

O SQL Server suporta três modos — conhecer as diferenças evita surpresas em produção.

3 MODOS · 9 ASPECTOS

DO DEFAULT AO PROFISSIONAL

ASPECTO Aspecto	DEFAULT Autocommit	ANSI Implícita	PRODUÇÃO Explícita
Comportamento	Cada statement é uma transação	BEGIN TRAN automático antes de cada DML	Você controla com BEGIN/COMMIT/ROLLBACK
Quando usar	Scripts ad-hoc, dev	Padrão ANSI, compatibilidade	Produção, lógica de negócio multi-tabela
BEGIN explícito?	Não (automático)	Não (mas acontece)	Sim, obrigatório
COMMIT explícito?	Não (automático)	Sim, ou próximo DML	Sim, obrigatório
Performance	✓ Rápido (1 statement)	⚠ Overhead por DML	✓ Controle fino
Recuperação em erro	Automática (rollback do statement)	Manual ou próxima instrução	Manual via TRY/CATCH
Ativação	Default do SQL Server	SET IMPLICIT_TRANSACTIONS ON	BEGIN TRANSACTION
Risco	Múltiplos statements não-atômicos	Esquecer COMMIT — locks eternos	Esquecer COMMIT/ROLLBACK no CATCH
Recomendado para	DDL, scripts de manutenção	Compatibilidade com padrão ANSI	TODA lógica de negócio transacional

✓ Regra prática: em produção, SEMPRE usar modo Explícito com TRY/CATCH + SET XACT_ABORT ON.

CONTROLE AVANÇADO 5.1.5

SAVEPOINT e Rollback Parcial

SAVE TRANSACTION cria um marcador dentro de uma transação, permitindo ROLLBACK parcial — desfazer apenas operações após o SAVEPOINT, mantendo as anteriores. Útil quando há etapas opcionais ou de risco.

SINTAXE

- `SAVE TRANSACTION nome` — cria ponto de salvamento
- `ROLLBACK TRANSACTION nome` — desfaz até o SAVEPOINT (não a transação inteira)
- Pode haver múltiplos SAVEPOINTs na mesma transação

CASOS DE USO

- Operações opcionais (log, notificação) que não devem abortar a transação principal principal
- Tentativa de operação com fallback (ex: tentar via SP, se falhar usar alternativa)
- Validação em lote — processar N itens, descartar apenas os que falharam
- Auditoria falha não deve abortar operação principal

LIMITAÇÕES

- Não pode ser usado fora de uma transação
- ROLLBACK ao SAVEPOINT não libera locks da transação
- Não conta para @@TRANCOUNT aninhada

✦ USP_PROCESSARPEDIDO.SQL — T-SQL · 66 linhas

```sql
-- Processar pedido com SAVEPOINT para auditoria opcional
CREATE OR ALTER PROCEDURE dbo.usp_ProcessarPedido
@PedidoId INT
AS
BEGIN
SET NOCOUNT ON;
SET XACT_ABORT ON;
BEGIN TRY
BEGIN TRAN;
-- 1. Operação principal (sempre deve funcionar)
UPDATE dbo.Pedidos
SET Status = 'PROCESSADO', DataProcessamento = GETDATE()
WHERE PedidoId = @PedidoId;
IF @@ROWCOUNT = 0
THROW 50001, 'Pedido não encontrado', 1;
-- 2. Operação crítica: atualizar estoque
UPDATE p
SET p.Estoque = p.Estoque - ip.Quantidade
FROM dbo.Produtos p
INNER JOIN dbo.ItensPedido ip
ON ip.ProdutoId = p.ProdutoId
WHERE ip.PedidoId = @PedidoId;
-- 3. SAVEPOINT: operação de auditoria opcional
-- Se falhar (ex: tabela de log cheia), NÃO abortar
SAVE TRAN AuditarPedido;
BEGIN TRY
INSERT INTO dbo.AuditoriaPedidos
(PedidoId, Usuario, DataProcessamento, Detalhes)
SELECT @PedidoId, SUSER_SNAME(),
GETDATE(), 'Pedido processado'
-- Pode falhar sem abortar
END TRY
BEGIN CATCH
-- Rollback apenas até SAVEPOINT
ROLLBACK TRAN AuditarPedido;
-- Logar falha da auditoria separadamente
PRINT 'Auditoria falhou: ' + ERROR_MESSAGE();
END CATCH
-- 4. Operação final: notificação (opcional)
SAVE TRAN NotificarCliente;
BEGIN TRY
-- Pode usar Service Broker para notificação assíncrona
PRINT 'Notificação enviada ao cliente';
END TRY
BEGIN CATCH
ROLLBACK TRAN NotificarCliente;
PRINT 'Notificação falhou (não crítico)';
END CATCH
-- Confirma transação principal
COMMIT TRAN;
PRINT 'Pedido processado com sucesso';
END TRY
BEGIN CATCH
IF XACT_STATE() <> 0 ROLLBACK TRAN;
THROW;
END CATCH
END
GO
```

CONCEITO AVANÇADO 5.1.6

Transações Aninhadas e @@TRANCOUNT

T-SQL suporta transações aninhadas (BEGIN TRAN dentro de BEGIN TRAN), MAS com comportamento não-trivial: apenas o COMMIT mais externo confirma a transação, e ROLLBACK sempre desfaz TODAS as alterações (não apenas o nível atual). Compreender este comportamento é essencial para SPs que chamam outras SPs transacionais.

REGRAS

- Cada BEGIN TRAN incrementa @@TRANCOUNT
- Cada COMMIT decrementa @@TRANCOUNT
- COMMIT só confirma fisicamente quando @@TRANCOUNT chega a 0
- ROLLBACK sempre zera @@TRANCOUNT e desfaz TUDO
- ROLLBACK ao SAVEPOINT não afeta @@TRANCOUNT

@@TRANCOUNT

0	sem transação ativa
1	transação ativa (mais comum)
2+	transações aninhadas

PADRÃO PROFISSIONAL EM SPS

- Verificar @@TRANCOUNT > 0 antes de ROLLBACK
- Usar SAVE TRANSACTION em vez de BEGIN TRAN aninhado
- Documentar se a SP inicia ou participa de transação

✦ NESTED_TRANSACTIONS.SQL — 2 SPs + demo

```sql
-- SP aninhada que participa de transação externa
CREATE OR ALTER PROCEDURE dbo.usp_AtualizarCliente
@ClienteId INT,
@NovoEmail VARCHAR(150)
AS
BEGIN
SET NOCOUNT ON;
DECLARE @TransacaoLocal BIT = 0;
-- Só inicia transação se não houver uma ativa
IF @@TRANCOUNT = 0
BEGIN
BEGIN TRAN;
SET @TransacaoLocal = 1;
END
BEGIN TRY
UPDATE dbo.Clientes
SET Email = @NovoEmail
WHERE ClienteId = @ClienteId;
-- Só faz COMMIT se a transação foi iniciada localmente
IF @TransacaoLocal = 1
COMMIT;
END TRY
BEGIN CATCH
-- Só faz ROLLBACK se a transação foi iniciada localmente
-- Se chamada dentro de outra, deixa o erro subir
IF @TransacaoLocal = 1 AND XACT_STATE() <> 0
ROLLBACK;
THROW;
END CATCH
END
GO
-- SP externa que chama múltiplas SPs transacionais
CREATE OR ALTER PROCEDURE dbo.usp_MigrarCliente
@ClienteId INT,
@NovoEmail VARCHAR(150)
AS
BEGIN
SET NOCOUNT ON;
SET XACT_ABORT ON;
BEGIN TRY
BEGIN TRAN Migracao;
-- Chama SP aninhada: @@TRANCOUNT = 1 (não inicia nova transação)
EXEC dbo.usp_AtualizarCliente @ClienteId, @NovoEmail;
-- @@TRANCOUNT ainda = 1
INSERT INTO dbo.LogMigracao
(ClienteId, DataMigracao, NovoEmail)
VALUES (@ClienteId, GETDATE(), @NovoEmail);
COMMIT TRAN Migracao; -- @@TRANCOUNT = 0
PRINT 'Migração concluída';
END TRY
BEGIN CATCH
IF XACT_STATE() <> 0 ROLLBACK;
THROW;
END CATCH
END
GO
-- Demonstração do comportamento não-intuitivo:
BEGIN TRAN; -- @@TRANCOUNT = 1
BEGIN TRAN; -- @@TRANCOUNT = 2
INSERT INTO T1 VALUES (1);
COMMIT; -- @@TRANCOUNT = 1 (não confirma!)
ROLLBACK; -- @@TRANCOUNT = 0, desfaz TUDO
-- Resultado: T1 está vazia
```

CENÁRIO AVANÇADO 5.1.7

Transações Distribuídas (MSDTC)

Quando uma transação precisa modificar dados em **múltiplos servidores** (ex: SQL Server A + SQL Server + SQL Server B + Oracle), é necessária uma transação distribuída coordenada pelo Microsoft Distributed Microsoft Distributed Transaction Coordinator (MSDTC).

COMO FUNCIONA

- MSDTC é um serviço Windows separado
- Usa Two-Phase Commit (2PC) protocol

P1 **Prepare:** cada servidor confirma que está pronto

P2 **Commit:** se todos confirmaram, commit; senão, rollback

QUANDO USAR

- Operações cross-server (linked servers)
- Integração com outros SGBDs (Oracle, DB2)
- Message Queue + Database (ex: MSMQ + SQL Server)
- Coordenação com sistemas transacionais externos

LIMITAÇÕES

- Performance: ~10x mais lento que transação local
- Configuração: requer MSDTC instalado e configurado
- Rede: sensível a latência e firewall
- Troubleshooting: complexo (in-doubt transactions)

ALTERNATIVAS MODERNAS

- Saga pattern (em microservices)
- Outbox pattern (garante consistência eventual)
- Event sourcing

DISTRIBUTED_TRANSACTION.SQL

linked server · 2PC

```sql
-- Habilitar transação distribuída
-- Requer: MSDTC instalado no servidor Windows
-- Cenário: atualizar Vendas no SQL Server A
-- e Estoque no SQL Server B (linked server)
-- 1. Configurar linked server (se ainda não existir)
EXEC sp_addlinkedserver
@server = N'ServidorEstoque',
@srvproduct = N'SQL Server';
GO
-- 2. Transação distribuída
CREATE OR ALTER PROCEDURE dbo.usp_RegistrarVendaDistribuida
@VendaId INT,
@ProdutoId INT,
@Quantidade INT
AS
BEGIN
SET NOCOUNT ON;
SET XACT_ABORT ON;
BEGIN TRY
-- Inicia transação distribuída
BEGIN DISTRIBUTED TRANSACTION;
-- Atualiza servidor local (Vendas)
UPDATE dbo.Vendas
SET Status = 'CONFIRMADA'
WHERE VendaId = @VendaId;
-- Atualiza servidor remoto (Estoque)
UPDATE [ServidorEstoque].EstoqueDB.dbo.Produtos
SET Estoque = Estoque - @Quantidade
WHERE ProdutoId = @ProdutoId;
-- COMMIT distribuído (2PC: Two-Phase Commit)
COMMIT TRANSACTION;
PRINT 'Venda registrada em ambos os servidores';
END TRY
BEGIN CATCH
IF XACT_STATE() <> 0
ROLLBACK TRANSACTION;
THROW;
END CATCH
END
GO
-- Verificar transações distribuídas em curso
SELECT *
FROM sys.dm_tran_active_transactions
WHERE transaction_type = 4; -- 4 = distributed
GO
-- Resolver transação 'in-doubt' (manual, raro)
-- (em caso de falha de rede entre prepare e commit)
-- EXEC sp_resolve_distributed_transaction
--   @xact_id, 'commit' | 'rollback';
```

EXEMPLO INTEGRADOR 5.1.8 · template de produção

Padrão Profissional com TRY/CATCH + XACT_STATE()

Este é o template que toda SP transacional em produção deveria seguir — robusto, com log estruturado, retry para deadlocks e tratamento granular de erros.

✓ CARACTERÍSTICAS DO TEMPLATE

- ✓ **SET XACT_ABORT ON** (auto-rollback em runtime)
- ✓ **TRY/CATCH estruturado**
- ✓ Verificação **XACT_STATE()** no CATCH
- ✓ Log estruturado em tabela dedicada
- ✓ **Retry para deadlocks** (error 1205)
- ✓ **THROW** para re-lançar ao cliente
- ✓ Nomes de transação (auditoria)
- ✓ **SET NOCOUNT ON**
- ✓ Validiação de parâmetros

✦ USP_TRANSFERENCIAPROFISSIONAL.SQL

86 linhas · retry + log

```
CREATE OR ALTER PROCEDURE dbo.usp_TransferenciaProfissional
@DeConta INT,
@ParaConta INT,
@Valor DECIMAL(18,2),
@MaxRetries INT = 3
AS
BEGIN
SET NOCOUNT ON;
SET XACT_ABORT ON;
DECLARE @Tentativa INT = 0;
DECLARE @Sucesso BIT = 0;
WHILE @Tentativa < @MaxRetries AND @Sucesso = 0
BEGIN
SET @Tentativa += 1;
BEGIN TRAN Transferencia;
-- Validação: saldo suficiente
IF NOT EXISTS (SELECT 1 FROM dbo.Contas
WHERE Conta = @DeConta
AND Saldo >= @Valor)
THROW 50001, 'Saldo insuficiente', 1;
-- Validação: contas diferentes
IF @DeConta = @ParaConta
THROW 50002, 'Contas origem e destino iguais', 1;
UPDATE dbo.Contas
SET Saldo = Saldo - @Valor,
DataUltimaAlteracao = GETDATE()
WHERE Conta = @DeConta;
UPDATE dbo.Contas
SET Saldo = Saldo + @Valor,
DataUltimaAlteracao = GETDATE()
WHERE Conta = @ParaConta;
INSERT INTO dbo.LogTransferencias
(DeConta, ParaConta, Valor, Usuario, DataHora)
VALUES (@DeConta, @ParaConta, @Valor,
SUSER_SNAME(), GETDATE());
COMMIT TRAN Transferencia;
SET @Sucesso = 1;
END TRY
BEGIN CATCH
-- Transação ativa, rollback
IF XACT_STATE() <> 0
ROLLBACK TRAN Transferencia;
-- Deadlock: tenta de novo
IF ERROR_NUMBER() = 1205 AND @Tentativa < @MaxRetries
BEGIN
-- Espera exponencial (backoff)
WAITFOR DELAY CONCAT('00:00:0',
POWER(2, @Tentativa) / 100.0);
END
ELSE
-- Outro erro: loga e re-lança
INSERT INTO dbo.LogErros
(Numero, Severidade, Mensagem, Procedure,
Linha, Tentativa, DataErro)
VALUES (ERROR_NUMBER(), ERROR_SEVERITY(),
ERROR_MESSAGE(), ERROR_PROCEDURE(),
ERROR_LINE(), @Tentativa, GETDATE());
-- Re-lança ao cliente
THROW;
END CATCH
END
IF @Sucesso = 1
PRINT 'Transferência concluída na tentativa ' +
CAST(@Tentativa AS VARCHAR);
END
GO
```

DEADLOCK VICTIM

Erro 1205 indica vítima de deadlock — backoff exponencial: 2^tentativa / 100s.

SEÇÃO

5.2

CONCURRENCY CONTROL

Controle de Concorrência, Níveis de Isolamento, Locks e Deadlocks

Quando múltiplos usuários acessam os mesmos dados — o coração operacional de qualquer de qualquer SGBD.

01 Concorrência: o problema
02 Tipos de locks
03 Matriz de compatibilidade
04 Níveis de isolamento
05 Snapshot (RCSI)
06 Deadlocks
07 Prevenção
08 Checklist profissional

Concorrência — Anomalias que Devemos Evitar

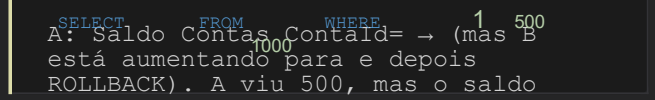
Seis problemas clássicos que ocorrem quando transações concorrentes acessam os mesmos dados **sem isolamento adequado**.

Dirty Read

01

Leitura Suja. Transação A lê dado modificado por B (ainda não (ainda não committed). Se B fazer ROLLBACK, A leu dado que dado que nunca existiu. `Solução: READ COMMITTED.`

EXEMPLO



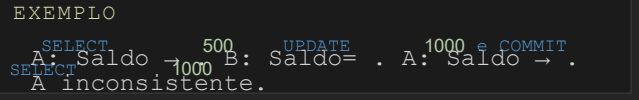
A: Saldo Contas Contaid= → (mas B está aumentando para e depois ROLLBACK). A viu 500, mas o saldo real era 1000.

Non-Repeatable Read

02

Leitura Não-Repeatível. A lê mesma linha duas vezes e obtém obtém valores diferentes porque B modificou entre as leituras. leituras. `Solução: REPEATABLE READ.`

EXEMPLO



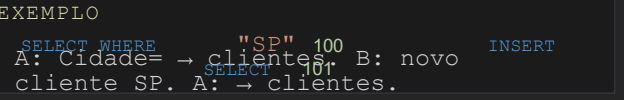
A: Saldo → B: Saldo= . A: Saldo → . A inconsistente.

Phantom Read

03

Leitura Fantasma. A executa mesma query duas vezes e obtém e obtém conjunto diferente de linhas porque B inseriu/deletou. `Solução: SERIALIZABLE.`

EXEMPLO



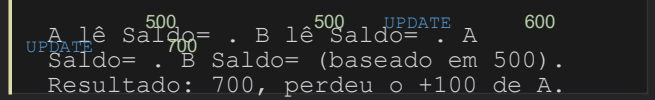
A: Cidade= → clientes. B: novo cliente SP. A: → clientes.

Lost Update

04

Atualização Perdida. A lê, B lê, A atualiza, B atualiza — atualização de A é perdida. `Solução: REPEATABLE READ READ ou UPDATE com WHERE otimista.`

EXEMPLO



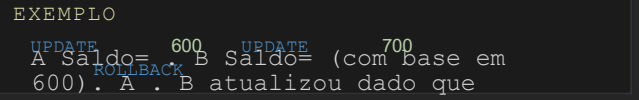
A lê Saldo= . B lê Saldo= . A Saldo= . B Saldo= (baseado em 500). Resultado: 700, perdeu o +100 de A.

Dirty Write

05

Escrita Suja. A atualiza, B atualiza antes de A commit, A faz faz ROLLBACK. B atualizou dado que não devia existir. `Solução: locks X impedem (SQL Server previne por default).`

EXEMPLO



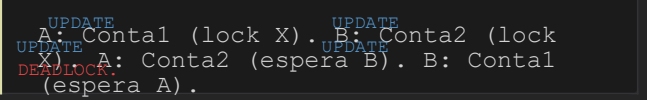
A Saldo= B Saldo= (com base em 600). A . B atualizou dado que não existia.

Deadlock

06

Impasse. A espera recurso bloqueado por B, B espera recurso recurso bloqueado por A. SQL Server escolhe vítima e aborta aborta uma. `Solução: ordenação consistente + retry.`

EXEMPLO



A: Conta1 (lock X). B: Conta2 (lock X). A: Conta2 (espera B). B: Conta1 (espera A).

◆ Cada anomalia é prevenida por um **nível específico de isolamento**. Vamos ver cada um.

Tipos de Locks e Matriz de Compatibilidade

O SQL Server usa diferentes tipos de locks dependendo da operação — conhecer a compatibilidade é essencial para diagnóstico

☰ SEÇÃO 1 — TIPOS DE LOCKS

8 tipos · granularidade row → database

Símbolo	Nome	Descrição	Quando usado
S	Shared (Compartilhado)	Permite leitura, bloqueia escrita	Operações SELECT (default em READ COMMITTED)
X	Exclusive (Exclusivo)	Bloqueia leitura e escrita	INSERT, UPDATE, DELETE
IS	Intent Shared	Indica lock S em nível inferior	Hierarquia: database → table → page → row
IX	Intent Exclusive	Indica lock X em nível inferior	Antes de UPDATE/DELETE
SIX	Shared with Intent Exclusive	S em resource, IX em sub-resources	Leitura + escrita na mesma tabela
U	Update	Pode evoluir para X	SELECT ... WITH UPDLOCK
Sch-S	Schema Stability	Lock de schema (estável)	Qualquer query em execução
Sch-M	Schema Modification	Modificação de schema	DDL: CREATE/ALTER/DROP

▦ SEÇÃO 2 — MATRIZ DE COMPATIBILIDADE (5×6)

✓ compatível · ✗ conflito

	I S		S	

Níveis de Isolamento do SQL Server

Cinco níveis tradicionais + 2 baseados em snapshot (MVCC) — escolha conforme trade-off entre consistência e concorrência

	READUNCOMMITTED	READ COMMITTED★ default	REPEATABLEREAD	SERIALIZABLE	SNAPSHOT	RCSI(Read Committed Snapshot)
Dirty Read	⚠ PERMITE	✓ Previne	✓ Previne	✓ Previne	✓ Previne	✓ Previne
Non-Repeatable Read	⚠ Permite	⚠ Permite	✓ Previne	✓ Previne	✓ Previne	⚠ Permite
Phantom Read	⚠ Permite	⚠ Permite	⚠ Permite	✓ Previne	✓ Previne	⚠ Permite
Lost Update	⚠ Permite	⚠ Permite	✓ Previne	✓ Previne	✓ Previne	⚠ Permite
Mecanismo	NOLOCK	Lock S, libera após ler	Lock S até fim da transação	Range locks	MVCC (versionamento)	MVCC para READ COMMITTED
Concorrência	✓✓ Máxima	✓ Alta	⚠ Média	✗ Baixa	✓ Alta	✓✓ Máxima
Performance	✓ Melhor	✓ Boa	⚠ Piora	✗ Pior	✓ Boa	✓✓ Ótima
Overhead	Nenhum	Baixo	Médio (locks)	Alto (range locks)	Versões em tempdb	Versões em tempdb
Deadlocks	✓ Evita	✗ Comum	✗ Comum	✗ Frequentes	✓ Evita	✓ Reduz
Recomendado	✗ Nunca (só relatório)	✓ Default	✗ Raro	✗ Raro (sistema crítico)	✓✓ Preferido	✓✓ Preferido

SQL Server 2019+: RCSI é recomendado como default (habilitar via `ALTER DATABASE SET READ_COMMITTED_SNAPSHOT ON`). Combina consistência com alta concorrência.

⚡ MVCC — ALTERNATIVA MODERNA

Snapshot Isolation (RCSI)

Snapshot Isolation usa Multi-Version Concurrency Control (MVCC) — quando uma transação lê uma linha, o SQL Server retorna a versão consistente do dado no momento em que a transação começou (ou a leitura ocorreu), sem precisar de locks S. Escritores não bloqueiam leitores, leitores não bloqueiam escritores.

☎ DUAS IMPLEMENTAÇÕES

Snapshot Isolation (SI) — requer `SET TRANSACTION ISOLATION LEVEL SNAPSHOT` na sessão

RCSI (Read Committed Snapshot Isolation) — substitui o comportamento default do READ COMMITTED para usar versionamento

⚙ COMO FUNCIONA

- Cada linha modificada gera versão em `tempdb`
- Leitores acessam versão apropriada (sem locks)
- Escritores ainda usam locks `X` entre si
- Conflito de escrita detectado (não bloqueia)

↗ VANTAGENS

- Elimina locks S em leituras
- Reduz drasticamente deadlocks
- Aumenta concorrência
- Mesma consistência que READ COMMITTED

⚠ DESVANTAGENS

- Overhead em `tempdb` (versões)
- Requer configuração no database
- Transações longas geram muitas versões

✦ rcsi_enable.sql

T-SQL · SQL Server 2019+

```sql
-- 1. Habilitar RCSI (substitui default do READ COMMITTED)
-- Requer acesso exclusivo ao database
ALTER DATABASE VendasDB
SET                     ON
WITH ROLLBACK IMMEDIATE-- derruba conexões ativas
GO

-- 2. Habilitar Snapshot Isolation (opcional)
ALTER DATABASE VendasDB
SET                     ON
GO

-- 3. A partir de agora, READ COMMITTED usa MVCC
-- (sem mudança no código da aplicação!)
SET TRANSACTION ISOLATION LEVEL READ COMMITTED default
BEGIN TRAN
SELECT      FROM          WHERE         1 -- sem lock S!
-- Outra transação pode UPDATE enquanto isto executa
COMMIT
GO

-- 4. Snapshot Isolation explícito (mais forte)
SET TRANSACTION ISOLATION LEVEL SNAPSHOT
BEGIN TRAN
-- Toda leitura vê versão consistente do início da transação
SELECT      FROM          WHERE         1
-- Mesmo que outra transação faça UPDATE e COMMIT,
-- esta leitura ainda vê o valor antigo
COMMIT
GO

-- 5. Conflito de escrita em SNAPSHOT
-- Transação A
SET TRANSACTION ISOLATION LEVEL SNAPSHOT
BEGIN TRAN
UPDATE          SET             100 WHERE         1
-- (não faz COMMIT ainda)

-- Transação B (concorrente)
SET TRANSACTION ISOLATION LEVEL SNAPSHOT
BEGIN TRAN
UPDATE          SET             50 WHERE          1
-- ERRO 3960: Snapshot update conflict
-- A transação B é abortada — deve tentar de novo
ROLLBACK
GO
```

Deadlocks — Causa e Diagnóstico

Ocorrem quando duas transações mantêm locks e cada uma espera um recurso recurso que a outra detém — ciclo mortal. O SQL Server escolhe uma vítima (erro vítima (erro 1205) e aborta-a para que a outra prossiga.

⊗ CAUSAS MAIS COMUNS

- Ordem inversa de acesso a recursos (clássico A→B vs B→A)
- Transações longas mantendo locks por tempo excessivo
- Locks em índices não-alinhados com a query
- Nível de isolamento muito restritivo (SERIALIZABLE)
- Ausência de índice → scan eleva chance de lock em X

⌕ DIAGNÓSTICO

- Extended Events `xml_deadlock_report` — gráfico XML completo
- Sessão `system_health` já captura por default
- `sys.dm_tran_locks` — locks em tempo real
- `sys.dm_os_waiting_tasks` — quem espera quem
- Error log: 1205 indica a transação vítima

! **Deadlock ≠ bloqueio.** Em bloqueio, a transação espera e prossegue; em deadlock, o ciclo nunca se resolve sozinho — o engine mata uma das partes.

✦ deadlock_diagnostic.sql · 5 blocos

9px · 1.2 lh

▦ 1 · Extended Events — captura — setup · one-time

```
-- Capturar deadlocks via Extended Events
CREATE EVENT SESSION Deadlock_Capture ON SERVER
ADD EVENT
    ACTION
ADD TARGET                  SET        N'C:\Temp\deadlocks.xel'
ALTER EVENT SESSION Deadlock_Capture ON SERVER STATESTART
```

⌕ 2 · Query — extrair o gráfico de deadlock — XQuery · system_health

```
SELECT            '(@timestamp)[1]'    'datetime2'    AS
                  '(data/value/deadlock)[1]'      AS
FROM SELECT CAST           AS     AS
     FROM
     JOIN                    ON
     WHERE        'system_health'
       AND                   'event_file' AS
CROSS APPLY                '//event[@name="xml_deadlock_report"]'
```

⊘ 3 · Anti-pattern — ordem inversa — DEADLOCK

```
-- Sessão A
BEGIN TRAN
UPDATE          SET              100 WHERE        1   -- lock X em Conta 1
UPDATE          SET              100 WHERE        2   -- espera Conta 2
COMMIT
-- Sessão B (concorrente) — ordem INVERSA
BEGIN TRAN
UPDATE          SET               50  WHERE       2   -- lock X em Conta 2
UPDATE dbo.Contas SET Saldo = Saldo + 50  WHERE ContaId = 1;  -- DEADLOCK → erro 1205
```

✓ 4 · Prevenção — mesma ordem — SAFE

```
-- A e B ambas seguem a MESMA ordem (Conta 1 → Conta 2)
BEGIN TRAN
UPDATE          SET              100 WHERE        1   -- lock X em Conta 1
UPDATE dbo.Contas SET Saldo = Saldo + 100 WHERE ContaId = 2;  -- apenas espera — sem deadlock
COMMIT
```

▶ 5 · Retry — tratamento transiente — TRY/CATCH

```
DECLARE @retries INT  0
WHILE @retries 3
BEGIN
    BEGIN TRY
       BEGIN TRAN
        UPDATE          SET              100 WHERE        1
        UPDATE          SET              100 WHERE        2
        COMMIT BREAK
    END TRY
   BEGIN CATCH
       IF ERROR_NUMBER    1205  -- deadlock victim
        BEGIN ROLLBACK SET @retries   1 WAITFOR DELAY '00:00:00.050'    END
       ELSE THROW
    END CATCH
END
```

Padrões de Prevenção de Deadlocks

Seis técnicas consagradas para evitar ciclos mortais — ordem, brevidade, versionamento, indexes, retry e isolamento mínimo.
isolamento mínimo.

6 PADRÕES · ✓ DEFENSIVO

PADRÃO 01

Ordem Consistente de Acesso

Acesse tabelas/recursos sempre na **mesma sequência** em todas as transações. Elimina o ciclo A→B vs B→A — causa mais comum de deadlock.

```
-- padronize: Conta 1 → Conta 2
UPDATE dbo.Contas SET ... WHERE ContaId = 1;
UPDATE dbo.Contas SET ... WHERE ContaId = 2;
```

★ ALTO IMPACTO

PADRÃO 02

Transações Curtas

Mantenha a transação **aberta pelo menor tempo possível**. possível. Nada de I/O externo, chamadas de rede ou input ou input do usuário entre BEGIN TRAN e COMMIT.

```
BEGIN TRAN;
-- só DML dentro da transação
UPDATE ...; INSERT ...;
COMMIT; -- libera locks X imediatamente
```

★ ALTO IMPACTO

PADRÃO 03

Versionamento (RCSI / SNAPSHOT)

Leitores não bloqueiam escritores — MVCC **elimina a elimina a maior parte dos deadlocks** de leitura/escrita. leitura/escrita. Azure SQL já vem habilitado por default. default.

```
ALTER DATABASE VendasDB
SET READ_COMMITTED_SNAPSHOT ON
WITH ROLLBACK IMMEDIATE;
```

★ ALTO IMPACTO

PADRÃO 04

Indexes Corretos & SEEK

TABLE SCAN eleva a chance de lock em X em milhares de milhares de linhas. Index SEEK acessa só as linhas necessárias — locks mais cirúrgicos, menos conflitos. conflitos.

```
-- evitar: scan gera muitos locks
SELECT * FROM Pedidos WHERE Total > 1000;
-- criar:
CREATE INDEX IX_Pedidos_Total ON dbo.Pedidos
```

◆ MÉDIO IMPACTO

PADRÃO 05

Retry com Backoff Exponencial

Mesmo com todas as técnicas, deadlocks residuais ocorrem. Trate ocorrem. Trate **erro 1205** com retentativas (3–5) e backoff exponencial — Polly, Resilience frameworks.

```
IF ERROR_NUMBER() = 1205
BEGIN
@retries += 1;
WAITFOR DELAY '00:00:00.050';
-- backoff: 50ms → 100ms → 200ms
END
```

◆ MÉDIO IMPACTO

PADRÃO 06

Isolamento Mínimo Necessário

Evite **SERIALIZABLE** — range locks travam faixas inteiras. inteiras. Use `READ COMMITTED` + `UPDLOCK` pontual quando precisar de read-then-update atômico.

```
-- lock cirúrgico em vez de range lock
BEGIN TRAN;
SELECT Saldo FROM dbo.Contas WITH (UPDLOCK)
WHERE ContaId = 1;
UPDATE dbo.Contas SET Saldo -= 100 ...;
COMMIT;
```

◆ MÉDIO IMPACTO

Monitoramento de Locks e Deadlocks

Visibilidade contínua é a base da operação saudável. DMVs e Extended Events Events permitem detectar bloqueios, identificar head-blockers e analisar deadlocks deadlocks históricos sem precisar reproduzir o problema.

DMVS PRINCIPAIS

`sys.dm_tran_locks`	locks ativos no instante instante
`sys.dm_os_waiting_tasks`	sessões em espera
`sys.dm_exec_requests`	requests em execução
`sys.dm_os_wait_stats`	wait stats acumulados
`sys.dm_tran_active_transactions`	transações em andamento

SINAIS DE PROBLEMA

- Wait type LCK_M_* dominante no top waits
- blocking_session_id <> 0 por mais de 30s
- Head blocker durando > 60s — candidatar a KILL
- Aumento de `deadlocks/min` no baseline
- Transações abertas há >5 min (esquecidas)

Alerta recomendado: crie Job Agent / Extended Event que dispare quando quando wait_time_ms > 30000 em `LCK_M_IX`.

monitoring_queries.sql · 5 blocos

9px · 1.2 lh

1 · Locks correntes — quem trava o quê

real-time

```
SELECT
                    AS              AS

FROM
WHERE                 DB_ID'VendasDB'
  AND           IN  'OBJECT' 'PAGE' 'KEY'
ORDER BY
```

2 · Cadeia de espera — head blocker

real-time

```
SELECT          AS
                  AS

      SUBSTRING                        2 1  100  AS
FROM
LEFT JOIN                 ON
OUTER APPLY
WHERE         LIKE 'LCK%'
ORDER BY                DESC
```

3 · Wait stats — top locks acumulados

since startup

```
SELECT TOP10

      CAST          1.0 NULLIF              0  AS DECIMAL10 2  AS

FROM
WHERE       LIKE 'LCK%'
ORDER BY         DESC
```

4 · Deadlocks históricos — system_health

XEvent · XML

```
SELECT          '(@timestamp)[1]'   'datetime2'    AS
                '(data/value/deadlock/process-list/process[1]'
                '(data/value/deadlock/victim-list/victimProcess/@id)[1]'  'varchar(20)'    AS
FROM SELECTCAST          AS     AS
     FROM
     JOIN                  ON
     WHERE        'system_health'
       AND              'event_file'AS
CROSS APPLY             '//event[@name="xml_deadlock_report"]'
ORDER BY         DESC
```

5 · Transações abertas > 5 min

forgotten tx

```
SELECT

      DATEDIFFSECOND                        SYSUTCDATETIME AS
      COUNT              AS
FROM
JOIN                            ON
JOIN                   ON
LEFT JOIN                ON
GROUP BY
HAVINGDATEDIFFSECOND                      SYSUTCDATETIME   300
ORDER BY       DESC
```

Checklist Profissional + Síntese

12 práticas consagradas para sistemas transacionais saudáveis no SQL Server.

✓ 12 ITENS · PRONTO PARA PRODUÇÃO

01 ACID em toda transação
Garanta Atomicidade, Consistência, Isolamento e Isolamento e Durabilidade em cada operação composta.

02 BEGIN/COMMIT explícito
Nunca confie em autocommit para operações multi-statement — use `BEGIN TRAN ... COMMIT`.

03 Escolher isolamento consciente
Saiba o trade-off de cada nível — default nem nem sempre é o certo para seu workload.

04 Habilitar RCSI em produção
`SET READ_COMMITTED_SNAPSHOT ON` — Azure já vem, on-prem é manual.

05 Evitar NOLOCK
Restrinja `NOLOCK` a relatórios aproximados — nunca em OLTP crítico.

06 Ordem consistente de acesso
Padronize a sequência de tabelas em todas as transações que tocam os mesmos recursos.

07 Transações curtas
Sem I/O externo, chamadas de rede ou input do do usuário entre `BEGIN` e `COMMIT`.

08 Indexes com SEEK
TABLE SCAN eleva locks em X para milhares de de linhas — SEEK mantém locks cirúrgicos.

09 Retry no erro 1205
Trate deadlock victim com 3-5 retentativas e backoff backoff exponencial (Polly / resilience).

10 Monitorar DMVs continuamente
`dm_tran_locks`, `dm_os_waiting_tasks`, `dm_os_wait_stats` no dashboard.

11 Capturar deadlocks
Sessão `system_health` já grava — extraia via XQuery quando preciso.

12 Documentar padrões no time
Padronize convenções de transação, retry e isolamento no repositório de padrões do time. time.

◆ SÍNTESE DA AULA

- **Transações** garantem ACID — use-as explicitamente em operações operações compostas.
- **Locks** são o mecanismo de isolamento; conhecer tipos & compatibilidade é essencial.
- **Isolamento** é trade-off entre consistência e concorrência — RCSI é o ponto ótimo.
- **Deadlocks** são inevitáveis em sistemas concorrentes — trate com retry e prevenção.
- **Monitoramento** contínuo via DMVs e Extended Events é o coração da operação saudável.

→ PRÓXIMOS PASSOS

- ✓Aula 06 — Índices Avançados: columnstore, índices filtrados & filtrados & store procedures otimizadas.
- ✓Laboratório — Reproduzir deadlock controlado no AdventureWorks.
- ✓Leitura — Microsoft Docs: Transaction Locking and Row Versioning Guide.

✦ DESAFIO.sql — entrega até próxima aula

```
-- Desafio: habilitar RCSI + simular
-- 2 sessões concorrentes sem deadlock
ALTER DATABASE VendasDB
SET                    ON
WITH ROLLBACK IMMEDIATE
GO
-- Em 2 sessões: mesma ordem de Conta 1 → 2
BEGIN TRAN
UPDATE          SET         100
  WHERE         1
UPDATE          SET         100
  WHERE         2
COMMIT
```